My Project

Generated by Doxygen 1.16.1

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 3

Class Documentation

3.1 Command Struct Reference

Structure of three key elements, that are used for storing data passed in the terminal.

`#include <command.h>`

Public Attributes

- char * **file_name**
  Used for storing file's name.
- enum algorithm_type **chosen_algorithm**
  Used for storing algorithm's type.
- enum file_output_type **chosen_format**
  Used for storing desired format of the output file.

3.1.1 Detailed Description

Structure of three key elements, that are used for storing data passed in the terminal.

The documentation for this struct was generated from the following file:

- include/command.h

3.2 Edge Struct Reference

Structure, which contains graph partitioned into edges.

`#include <graph.h>`

Public Attributes

- char **edge_name** [2]

  Name of the edge (e.g. AB).
- unsigned int **start_node**

  Ordinal number of the edge's starting node.
- unsigned int **end_node**

  Ordinal number of the edge's ending node.
- double **weight**

  Edge's weight.
- struct Edge * **next**

  Pointer to the next edge.

3.2.1 Detailed Description

Structure, which contains graph partitioned into edges.

The documentation for this struct was generated from the following file:

- include/graph.h

3.3 Node Struct Reference

Structure, which contains graph partitioned into vertices.

```
#include <graph.h>
```

Public Attributes

- unsigned int **node**

  Ordinal number of the node.
- double **x_axis**

  Node's position in the x axis.
- double **y_axis**

  Node's position in the y axis.
- struct Node * **next**

3.3.1 Detailed Description

Structure, which contains graph partitioned into vertices.

The documentation for this struct was generated from the following file:

- include/graph.h

Chapter 4

File Documentation

4.1 include/command.h File Reference

Structure, which contain data gathered from terminal and enums, which contain a list of format types and algorithms.

Classes

- struct Command

  Structure of three key elements, that are used for storing data passed in the terminal.

Enumerations

- enum algorithm_type { TRIANGULATION , SPECTRAL }

  Set of algorithms' types.
- enum file_output_type { TEXT , BINARY }

  Set of file format types.

4.1.1 Detailed Description

Structure, which contain data gathered from terminal and enums, which contain a list of format types and algorithms.

Structure Command is being used for storing information, passed into the terminal. Enum is being used as a compact storage for algorithms and format types.

Warning

Although you can widen enum/structure, NEVER modify existing parameters niether in enums nor in the structure!

4.1.2 Enumeration Type Documentation

4.1.2.1 algorithm_type

`enum algorithm_type`

Set of algorithms' types.

Enumerator

TRIANGULATION	Used for highlighting chosen algorithm type, in our case triangulation algorithm.
SPECTRAL	Used for highlighting chosen algorithm type, in our case spectral algorithm.

4.1.2.2 file_output_type

`enum file_output_type`

Set of file format types.

Enumerator

TEXT	Used for highlighting chosen format type, in our case txt format.
BINARY	Used for highlighting chosen format type, in our case binary format.

4.2 command.h

Go to the documentation of this file.

```
00001
00010
00011 #ifndef COMMAND_H
00012 #define COMMAND_H
00013
00018 enum algorithm_type
00019 {
00020     TRIANGULATION,
00021     SPECTRAL
00022 };
00023
00028 enum file_output_type
00029 {
00030     TEXT,
00031     BINARY
00032 };
00033
00038 typedef struct
00039 {
00040     char *file_name;
00041     enum algorithm_type chosen_algorithm;
00042     enum file_output_type chosen_format;
00043 } Command;
00044
00045 #endif // COMMAND_H
```

4.3 include/command_executor.h File Reference

Function, which purpose is to combine other functions.

```
#include "../include/command.h"
#include "../include/io.h"
```

Functions

- int **execute_command** (Command cmd)

4.3.1 Detailed Description

Function, which purpose is to combine other functions.

4.4 command_executor.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef COMMAND_EXECUTOR_H
00009 #define COMMAND_EXECUTOR_H
00010 #include "../include/command.h"
00011 #include "../include/io.h"
00012
00013 int execute_command(Command cmd);
00014
00015 #endif // COMMAND_EXECUTOR_H
```

4.5 include/command_parser.h File Reference

Parse function, which gets data from the terminal.

```
#include "../include/command.h"
```

Functions

- int parse_command (int argc, char *argv[ ], Command *cmd)

  Function scans command terminal for file's name, algorithm type and file output type.

4.5.1 Detailed Description

Parse function, which gets data from the terminal.

Function is being used to get initial data, that is necessary for program to work.

Attention

Program highly depends on order, in which arguments were typed in. Also it heavily depends on flags and the way user typed them.

Note

To learn the way to properly type in data type /program help or read our user's manual

4.5.2 Function Documentation

4.5.2.1 parse_command()

```
int parse_command (
            int argc,
            char * argv[],
            Command * cmd)
```

Function scans command terminal for file's name, algorithm type and file output type.

In the process of scanning terminal function detects flags and process them.

Valid input will look like this: ./name_of_the_program <file_name> -a <algorithm_type> -t/-b

Flags meaning: -a - precedes <algorithm_type>. By default <algorithm_type> is triangulation. -t - output file has txt format. -b - output file has binary format. By default file has text format.

Parameters

argc	The number of input arguments.
argv	Array of the input arguments.
cmd	Structure elements, that are used for storing data passed in the terminal.

Returns

0 or error's id.

4.6 command_parser.h

Go to the documentation of this file.

```
00001
00010
00011 #ifndef FLAG_PARSER_H
00012 #define FLAG_PARSER_H
00013 #include "../include/command.h"
00014
00036 int parse_command(int argc, char *argv[], Command *cmd);
00037
00038 #endif // FLAG_PARSER_H
```

4.7 include/error_handler.h File Reference

Errors' constants, their definition and function(handle_error) that acts accordingly to the error type.

Macros

Standard Error Codes.

Initializing Errors' IDs definition for functions that can fail.

- #define **ERROR_INVALID_PARAMETER** (1)
  Invalid argument passed.
- #define **ERROR** (2)
  to be continued.

Functions

- void **handle_error** (int error_id)

4.7.1 Detailed Description

Errors' constants, their definition and function(handle_error) that acts accordingly to the error type.

Every error that can be present in our program is listed here. Every error has its unique id (e.g 1, 2, 25). IDs from 1-10 correspond to general and common errors. IDs that start from digit 2 correspond to errors related to files.

Attention

As the project grows, more errors appear. When they do, make sure that you use this list and add every single error here. When you add new error constants, don't forget to add description to them.

Warning

NEVER USE RAW NUMBERS AS ERROR TYPES!

4.8 error_handler.h

Go to the documentation of this file.

```
00001
00014
00015 #ifndef ERROR_HANDLER_H
00016 #define ERROR_HANDLER_H
00017
00024 #define ERROR_INVALID_PARAMETER (1)
00025 #define ERROR (2)
00027
00028 void handle_error(int error_id);
00029
00030 #endif // ERROR_HANDLER_H
```

4.9 include/graph.h File Reference

Structures, which contain graph broken down into parts.

Classes

- struct Edge

  Structure, which contains graph partitioned into edges.
- struct Node

  Structure, which contains graph partitioned into vertices.

4.9.1 Detailed Description

Structures, which contain graph broken down into parts.

Structure Edge: Initial graph is broken down into edges, that's why this structure is a singly linked list, where every single structure contains data about one edge. Structure Node: Generated graph is broken down into nodes, that's why this structure is a singly linked list, where every single structure contains data about one node.

4.10 graph.h

Go to the documentation of this file.

```
00001
00010
00011 #ifndef GRAPH_H
00012 #define GRAPH_H
00013
00018 typedef struct
00019 {
00020     char edge_name[2];
00021     unsigned int start_node;
00022     unsigned int end_node;
00023     double weight;
00024     struct Edge *next;
00025 } Edge;
00026
00031 typedef struct
00032 {
00033     unsigned int node;
00034     double x_axis;
00035     double y_axis;
00036     struct Node *next;
00037 } Node;
00038
00039 #endif // GRAPH_H
```

4.11 include/io.h File Reference

All file manipulations are done in here.

```
#include "../include/graph.h"
```

Functions

- int deserialize_file (char *filename, Edge *elements)

  Function opens file, checks validity (using check_file function), reads data in the file and saves it.
- FILE * create_output_file (enum file_output_type format, Node elements)

  Create an output file.

4.11.1 Detailed Description

All file manipulations are done in here.

Function for file deserialization and function for creating output file.

4.11.2 Function Documentation

4.11.2.1 create_output_file()

```
FILE * create_output_file (
            enum file_output_type format,
            Node elements)
```

Create an output file.

Parameters

format	desired format type of the output file.
elements	structure, containing data, which should be represented in the output file

Returns

File containing graph with coordinates.

4.11.2.2 deserialize_file()

```
int deserialize_file (
            char * filename,
            Edge * elements)
```

Function opens file, checks validity (using check_file function), reads data in the file and saves it.

Parameters

filename	name of the user's file.
elements	structure, into which data from the file is saved.

Returns

0 or error's id.

4.12 io.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef IO_H
00009 #define IO_H
00010 #include "../include/graph.h"
00011
00019 int deserialize_file(char *filename, Edge *elements);
00020
00028 FILE *create_output_file(enum file_output_type format, Node elements);
00029
00030 #endif // IO_H
```

4.13 spectral.h

```
00001 #ifndef SPECTRAL_H
00002 #define SPECTRAL_H
00003 #include "../include/graph.h"
00004
00005 int use_spectral_for_graph(Edge initial_graph, Node *output_graph);
00006
00007 #endif
```

4.14 triangulation.h

```
00001 #ifndef TRIANGULATION_H
00002 #define TRIANGULATION_H
00003 #include "../include/graph.h"
00004
00005 int triangulate_graph(Edge initial_graph, Node *output_graph);
00006
00007 #endif
```

Index